Assignment 4 Facade Pattern

Astana IT University | ShP-2216 Software Design Patterns | 2026-2027

Student Sabyr Aslan **Group** SE-2528 **Topic** Option A Media File Conversion

Repository https://github.com/asikxz008/Assigment4-sdp-Aslan

1 Introduction

The application converts real WAV and AU audio into mono audio through one public operation. Without a facade, a client must choose an output profile, validate the input header, decode PCM bytes, mix channels and write a container in the right order. MediaConversionFacade coordinates these steps while keeping their implementations in five independent subsystem classes [1, 2].

```
MediaConversionFacade converter = new MediaConversionFacade();
Path output = converter.convert("samples/stereo.wav", "au");
```

The client supplies a filename and target format and receives a standard Path. It handles ConversionException without knowing AudioFormat, byte order, sample arrays or temporary files.

2 Subsystem responsibilities

Class	Actual responsibility
CodecFactory	Selects WAV/little-endian or AU/big-endian output profile.
MetadataReader	Reads and validates container, PCM width, channels, rate and frames.
PcmDecoder	Decodes interleaved bytes into signed short samples; checks payload length.
AudioMixer	Averages the channels of each frame to produce mono samples.
AudioEncoder	Encodes PCM, writes the header and publishes a completed file.

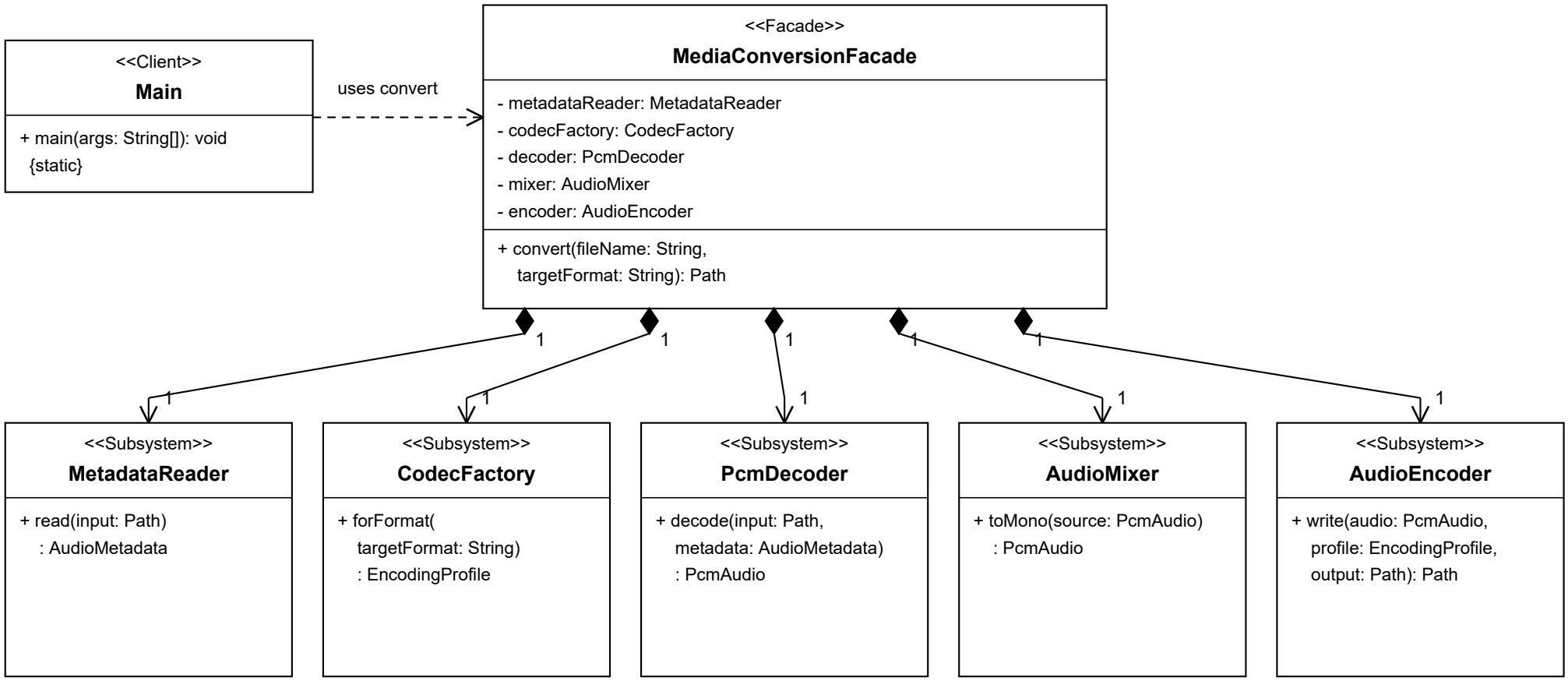
The facade invokes these steps in the table order. MetadataReader returns AudioMetadata to the decoder; the decoder returns PcmAudio to the mixer; the encoder consumes mixed PcmAudio and the factory's EncodingProfile. These shared values make the pipeline cohesive.

3 Concrete conversion example

A stereo frame with samples (-12000, 4000) becomes -4000. The four-frame fixture (-12000, 4000), (16000, 8000), (-32768, 32767), (6, 2) becomes [-4000, 12000, 0, 4]. WAV and AU store the same output samples with different container headers and byte order. Sample rate and frame count remain unchanged, so duration is preserved.

4 UML class diagram

Main is the application client. All other pattern participants are in com.aitu.sdp.media. Public operations match the Java implementation. The advanced-client check T10 uses the subsystems directly; Main uses only the facade.



Dashed arrow: client dependency. Filled diamond and solid arrow: owned collaborator with navigability.

Each facade owns one instance of each subsystem. There are no arrows from subsystems back to the facade.

Signatures name data carriers; AudioMetadata, PcmAudio, EncodingProfile and ConversionException are omitted.

5 Clean Code evidence

The following annotated excerpts are taken from the submitted source. Each principle explains a concrete design choice and its effect on this conversion workflow.

5 1 Meaningful names

```
Path output = converter.convert(args[0], args[1]);
```

src/Main.java:13

MediaConversionFacade, convert and output communicate the client's task. Subsystem names identify their steps, making the coordination sequence easy to read.

5 2 Single Responsibility

```
PcmAudio decoded = decoder.decode(input, metadata);
PcmAudio mono = mixer.toMono(decoded);
Path output = outputPath(input, profile.extension());
```

src/com/aitu/sdp/media/MediaConversionFacade.java:23

The facade coordinates decoding, mixing and output selection. It contains no sample decoding or averaging algorithm; those responsibilities remain in PcmDecoder and AudioMixer.

5 3 Small focused methods

```
writeContainer(audio, profile, temporary);
publish(temporary, destination);
```

src/com/aitu/sdp/media/AudioEncoder.java:21

writeContainer handles the audio bytes and header; publish handles the filesystem move. Separate methods keep these concerns short and readable within the encoder.

5 4 No subsystem details in the public interface

```
public Path convert(String fileName, String targetFormat) throws ConversionException {
```

src/com/aitu/sdp/media/MediaConversionFacade.java:15

The public API uses two strings, a Path and one failure type. The client does not accept or construct AudioMetadata, EncodingProfile or PcmAudio; private fields hide collaborators.

5 5 Error handling retains the original cause

```
throw new ConversionException("Cannot convert '" + fileName + "' to '"
        + targetFormat + "': " + exception.getMessage(), exception);
```

src/com/aitu/sdp/media/MediaConversionFacade.java:28

Coordination failures become one checked exception with input and target context. The original cause remains available for diagnosis. The encoder closes streams and removes temporary files.

5 6 Encapsulation protects sample data

```
public short[] samples() {
    return samples.clone();
}
```

src/com/aitu/sdp/media/PcmAudio.java:36

The constructor and accessor copy the array. An outside caller cannot change previously decoded audio through its original array or a returned array; T14 checks both cases.

6 Verification

Compiled using javac 25.0.1 with --release 17, -Xlint:all and -Werror; executed on Java 17.0.16. Only JDK classes are required. ConversionTests reads output headers and PCM bytes independently through Java Sound rather than trusting success messages [3].

Checks	Verified behavior	Result
T1-T3	WAV -> WAV, WAV -> AU and AU -> WAV; exact mono samples, rate and frames.	PASS
T4-T5	Original bytes unchanged; target names accept case and surrounding spaces.	PASS
T6-T9	Unsupported format, missing input, malformed file and truncated PCM fail.	PASS
T10	An advanced client invokes all five subsystems directly and writes correct AU.	PASS
T11	Publication failure preserves existing data and leaves no temporary file.	PASS
T12-T14	Mono/repeated conversion, argument validation and sample-array protection.	PASS

```
SUMMARY: 14/14 PASS
SUCCESS: stereo-converted.wav
SUCCESS: stereo-converted.au
```

The CLI demonstration also reports the expected unsupported mp3 target. Full run output is stored in tests-output.txt and demo-output.txt. For the live demonstration, compile from the repository root and run Main --demo, then ConversionTests.

7 Conclusion

MediaConversionFacade hides orchestration, validation, byte order, PCM decoding, mono mixing, container writing and error translation. Main needs one conversion call. The public subsystem classes remain directly accessible to advanced clients and never reference the facade; simplification does not remove their independent APIs.

The trade-off is a deliberately narrow convenience API: output is always mono signed 16-bit PCM, and the input must be mono/stereo WAV or AU with known rate and frame count. PCM payload is limited to 20 MiB. MP3, video, compression, resampling and streaming are outside scope. The encoder prefers an atomic move; its fallback regular move does not guarantee atomic visibility.

8 References

[1] Assignment 4 Facade Pattern Instructions. ShP-2216 Software Design Patterns, 2026-2027. Sections 2-5 and technical requirements table.

[2] Refactoring.Guru. Facade. https://refactoring.guru/design-patterns/facade. Accessed 14 October 2026.

[3] Oracle. Java SE 17 AudioSystem API. Java Sound API documentation. Methods getAudioFileFormat, getAudioInputStream and write. Accessed 14 October 2026.

[4] Source repository. https://github.com/asikxz008/Assigment4-sdp-Aslan. Incremental commits separate the subsystem, facade and checks, and report documentation.